

0.107.5. La fonction `teindreLutin()` — un lutin dans une palette

`#include <teindreLutin>` : `teindreLutin(numero, palette)` met le lutin dans une des 8 palettes des lutins (Game Boy Color).

1. Ce qu'il faut comprendre

Ce qu'elle fait : `teindreLutin(numero, palette)` met le lutin dans une autre **palette des lutins** : deux personnages avec le même dessin, de deux couleurs.

La ligne à écrire : `#include <teindreLutin>`.

Ses arguments : le numéro du lutin, la palette (0 à 7). À écrire **après** son `sprite()`.

Ce qu'elle coûte : environ **130 octets** avec la mise en place des lutins.

Essaie : une palette 2 verte, et un troisième A dedans.

On retrouve `teindreLutin` dans 3 leçons

[0.108](#), [104](#), [106](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **303 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	57 o	← la console, à l'allumage – la mise en route de la console
main()	97 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
RangerLutins	11 o	← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Tuiles	16 o	← CopierTuiles – 1 lettre sur 43
DonneesTransfert	10 o	← InstallerTransfert – la petite routine qui copie les lutins

Lettres gravées : A

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `Notes`

Fonctions du langage appelées :

<code>sprite()</code>	× 2	46 o sur place, dans <code>main()</code>
<code>couleurLutin()</code>	× 2	24 o sur place, dans <code>main()</code>
<code>teindreLutin()</code>	× 1	17 o sur place, dans <code>main()</code>
<code>image()</code>	× 1	6 o sur place, dans <code>main()</code> + <code>AttendreImage</code> (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

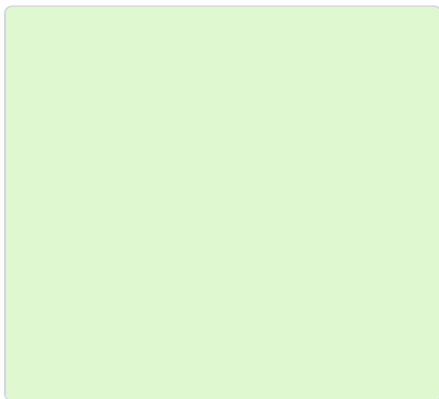
```
// ---- #include <teindreLutin> : un lutin dans une palette ----  
// Deux A : le premier dans la palette 0 (bleue), le second dans la 1 (rouge).  
  
#include <couleurLutin>  
#include <sprite>  
#include <teindreLutin>    // teindreLutin() : la palette d'un lutin  
#include <ALPHABET>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    couleurLutin(0, 3, 0, 0, 31);    // palette 0 : bleu  
    couleurLutin(1, 3, 31, 0, 0);    // palette 1 : rouge  
    sprite(0, 60, 68, ALPHABET[0]);  
    sprite(1, 92, 68, ALPHABET[0]);  
    teindreLutin(1, 1);              // le lutin 1 dans la palette 1  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Deux A côte à côte : un bleu, un rouge.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le lutin 0 est dans la palette 0

✓ le lutin 1 est dans la palette 1 (1)

Ensuite : **0.108. Une lettre qui tire — des étincelles** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++